Unit 1

1. What are Machine Learning Basics? Explain supervised and unsupervised learning.
2. Explain Capacity, Overfitting and Underfitting with a clear diagram.
3. What are Hyperparameters and what is a Validation Set?
4. What are Estimators? Explain Bias and Variance.
5. Explain Maximum Likelihood Estimation.
6. Explain Bayesian Statistics.
7. What is Stochastic Gradient Descent?
8. Explain the steps involved in Building a Machine Learning Algorithm.
9. What are Deep Feedforward Networks?
10. What is Gradient-Based Learning?
11. What is Backpropagation? Explain with a neat diagram.

1. What are Machine Learning Basics? Explain supervised and unsupervised learning.

1. **Machine Learning (ML)** means computers learn patterns automatically from data.
2. ML improves performance when more data is given.
3. ML is used in prediction, classification, recommendation, image recognition etc.

- **Supervised Learning:**

1. Data contains input + correct output (labels).
2. Model learns by comparing predicted answer with real answer.
3. Used for classification (spam/not spam) and regression (price prediction).

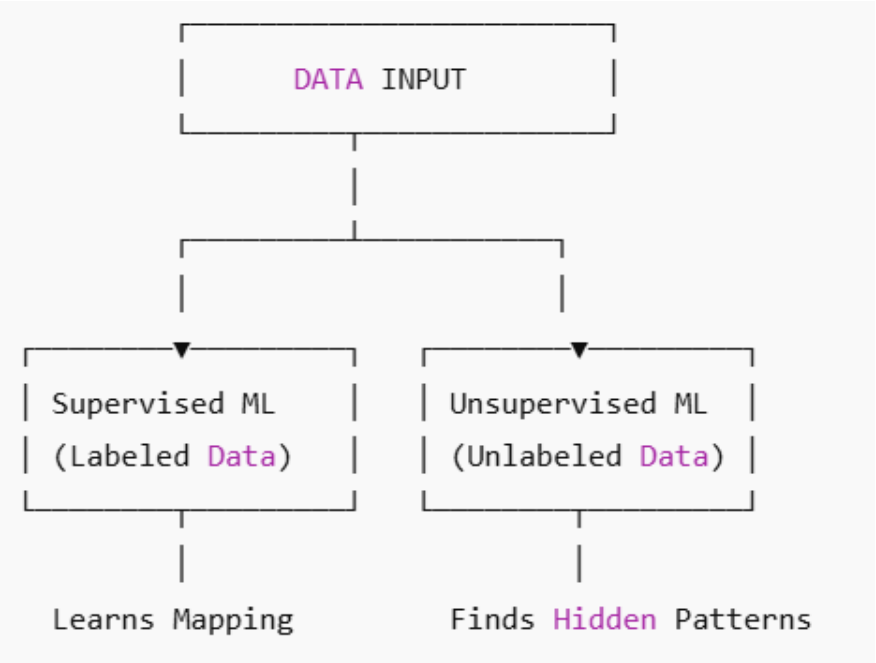
- Examples: Email spam detection, disease prediction.
  Supervised learning is divided mainly into **two types**:
  1) Classification
  Used when the output is a **category** or **label**.

1. Examples:
   – Email spam detection
   – Disease detection (positive/negative)
   – Handwritten digit recognition

Supervised Learning

2) Regression
Used when the output is a **continuous numeric value**.

Examples:
– House price prediction
– Temperature prediction
– Stock market value prediction



Unsupervised Learning:

1. Data has no labels (only input).
2. Model groups or discovers hidden patterns.
3. Useful when we don't know categories.
4. Examples: customer grouping, finding similar images.

In tabular form:

Supervised Learning	Unsupervised Learning
Data has **labels** (answers are known).	Data has **no labels** (answers are unknown).
Model learns using **input + correct output**.	Model learns by **finding patterns on its own**.
Goal: **Predict outputs** for new data.	Goal: **Discover hidden structures** in data.
Used for **classification and regression**.	Used for **clustering and dimensionality reduction**.
Training is more **complex** because labeled data is needed.	Training is **easy** because unlabeled data is sufficient.
High accuracy because of correct answers during training.	Lower accuracy since model learns by itself.
Examples: Spam vs Not Spam, Price prediction.	Examples: Grouping customers, Market segmentation.
Algorithms: Linear Regression, SVM, Decision Trees.	Algorithms: K-Means, PCA, Hierarchical Clustering.
Needs **large labeled datasets**, which is costly.	Works with **unlabeled datasets**, which are cheap.
Output is **known and measurable**.	Output is **unknown and discovered** by the model.

2. Explain Capacity, Overfitting and Underfitting with a clear diagram

- **Model Capacity:**

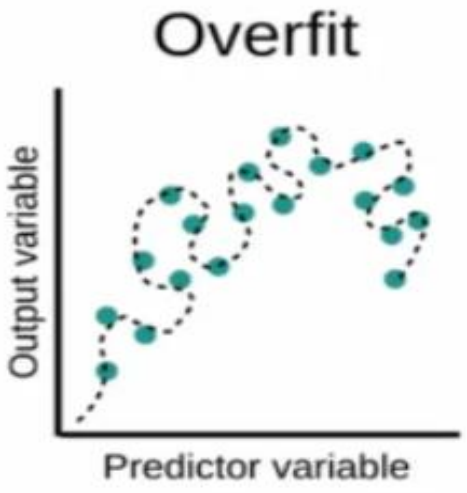
1. Capacity = how well a model can learn patterns.
2. Low capacity model learns few patterns.
3. High capacity model learns complex patterns.
4. Choosing correct capacity is very important.

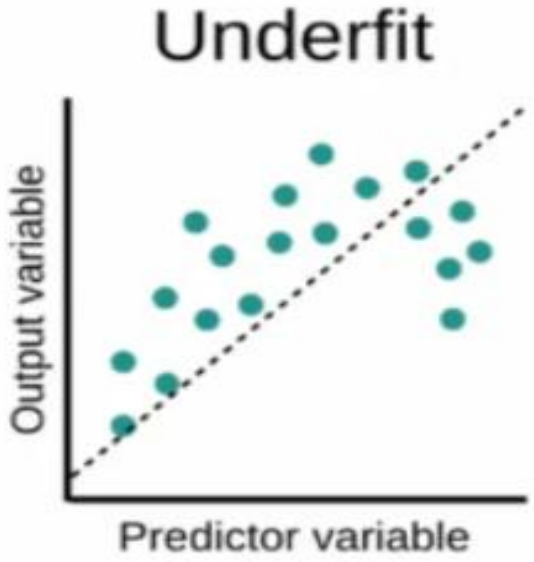
- **Overfitting:**

1. Model memorizes training data.
2. Performs well on training but poorly on new data.
3. Happens due to too many parameters / high capacity.
4. Example: Student memorizing answers but failing new questions.

- **Underfitting:**

1. Model cannot learn basic patterns.
2. Performs poorly on both training and test data.
3. Happens due to too simple model / low capacity.
4. Example: Student not studying enough.





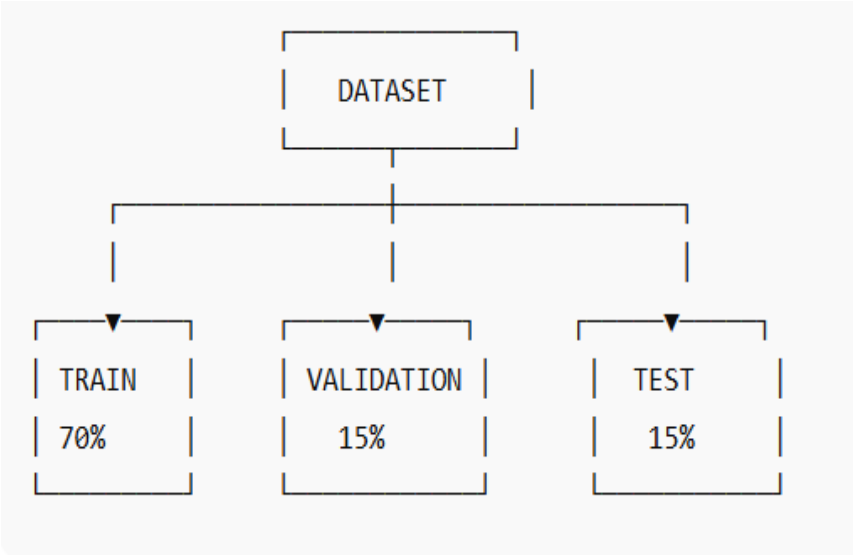
3. What are Hyperparameters and what is a Validation Set?

- **Hyperparameters:**

1. Values set before training (learning rate, layers, batch size).
2. They control learning speed and model behavior.
3. Not learned from data – manually chosen.

- **Validation Set:**

1. Small part of data used to tune hyperparameters.
2. Helps check performance during training.
3. Prevents overfitting and helps select best model.



→The full **dataset** is divided into 3 parts to train and check the model properly.

1. Training Set – 70%

1. Used to **train/teach** the model.
2. Model learns patterns from this data.

2. Validation Set – 15%

1. Used to **tune hyperparameters** (learning rate, layers, etc.).
2. Checks how well the model is performing during training.
3. Helps avoid **overfitting**.

3. Test Set – 15%

1. Used after training is completed.
2. Gives the **final accuracy** of the model.
3. Tells how well the model works on completely new data

Training = Learn

Validation = Improve

Test = Final performance check

4. What are Estimators? Explain Bias and Variance.

- **Estimators:**

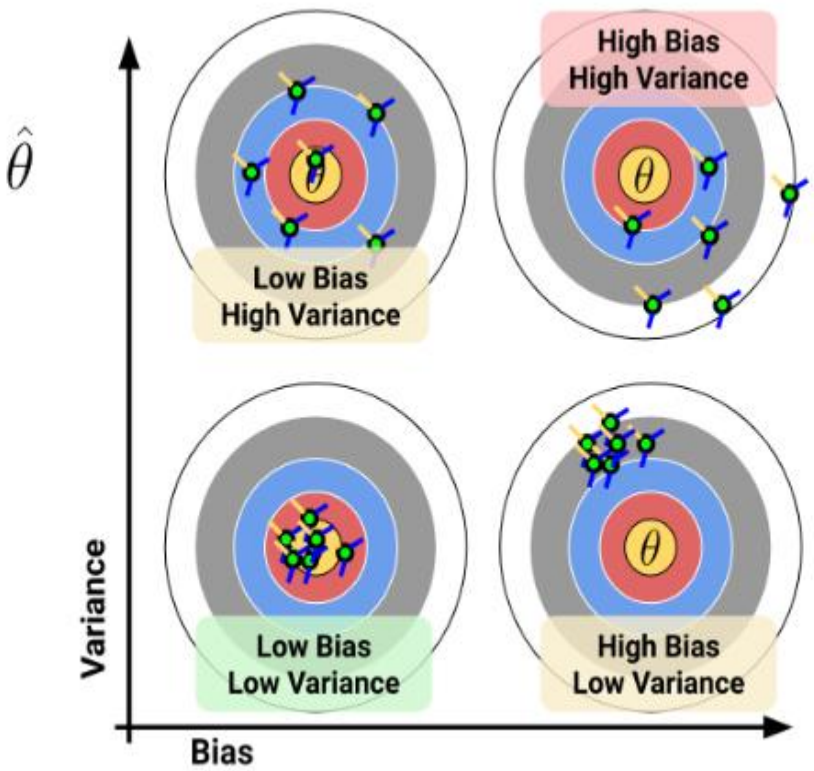
1. Algorithms that estimate values from data (mean, weights, parameters).
2. Used in regression, probability models etc.

- **Bias:**

1. Error caused by oversimplifying the model.
2. High bias → underfitting.
3. Model misses important patterns.

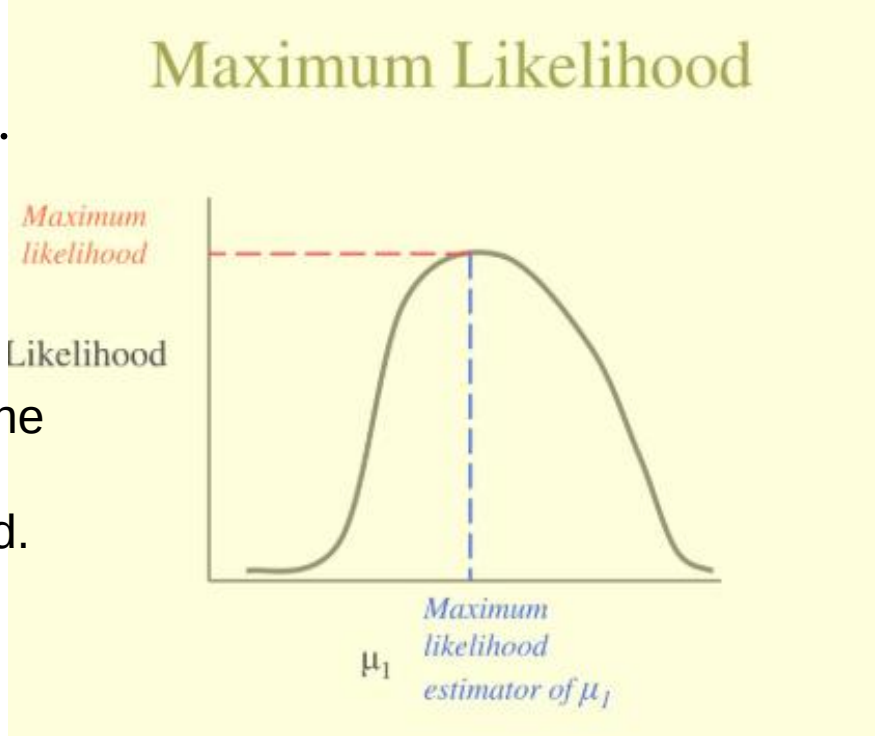
- **Variance:**

1. Error caused by high sensitivity to data noise.
2. High variance → overfitting.
3. Model memorizes data.



5. Explain Maximum Likelihood Estimation.

1. MLE finds parameters that best explain the given data.
2. It selects parameter values that make observed data most probable.
3. Used in regression, classification and probability models.
4. Example: estimating mean of normal distribution.



•**Purpose**: Estimate parameters that best explain the observed data.
•**Idea**: Pick parameter values that **maximize the likelihood** of the data.
•**Likelihood function**: L(θ) = P(data | θ), often use log-likelihood.

•**Steps**:
1.Write L(θ)
2.Take ln L(θ)
3.Differentiate w.r.t θ, set = 0
4.Solve for θ → MLE
•**Properties**: consistent, asymptotically normal, efficient
•**Example**: Coin toss n times, x heads → MLE of p = x/n

6. Explain Bayesian Statistics.

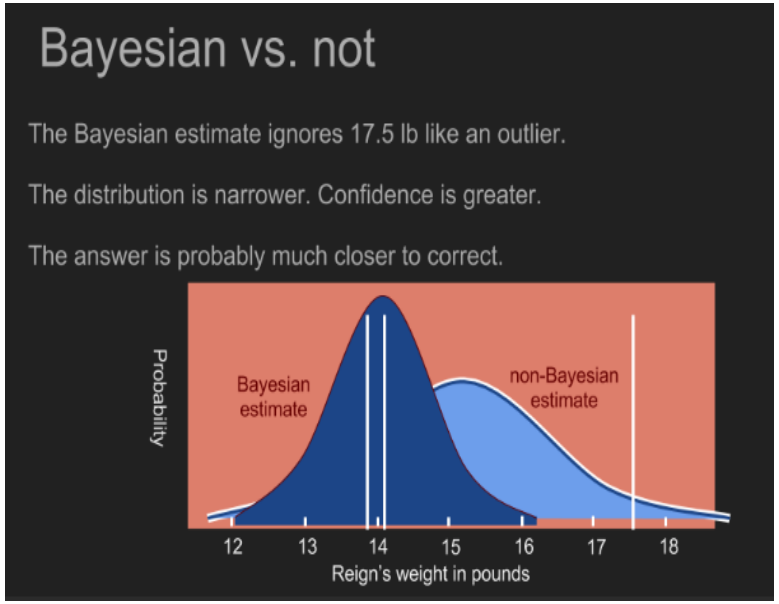
1. Bayesian method updates belief when new information comes.
2. Uses Bayes' theorem: Prior → evidence → posterior.
3. Helpful when data is uncertain or limited.
4. Used in spam filters, diagnosis, risk prediction.

•**Purpose**: Update beliefs about a parameter as new data becomes available.
•**Idea**: Combines **prior knowledge** with **observed data** to make inferences.
•**Bayes' Theorem**:
Posterior ∝ Likelihood × Prior
$P(\theta \mid data) = [P(data \mid \theta) \times P(\theta)] / P(data)$
•**Steps**:

- Choose a **prior** distribution for the parameter.
- Collect **data** and calculate the likelihood.
- Compute **posterior** distribution using Bayes' theorem.
- Make predictions or decisions based on the posterior.

•**Properties**:

- Incorporates prior knowledge
- Provides **full probability distribution** for parameters
- Naturally updates beliefs with new data

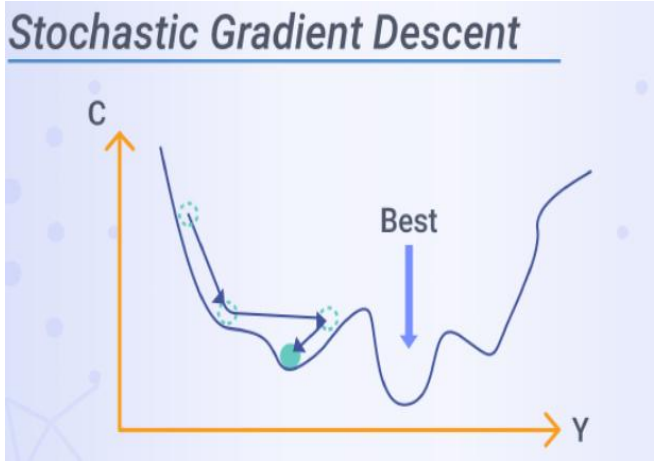


•**Example**: Estimating probability of rain using historical data (prior) and today's weather (data) → updated probability (posterior).

7. What is Stochastic Gradient Descent?

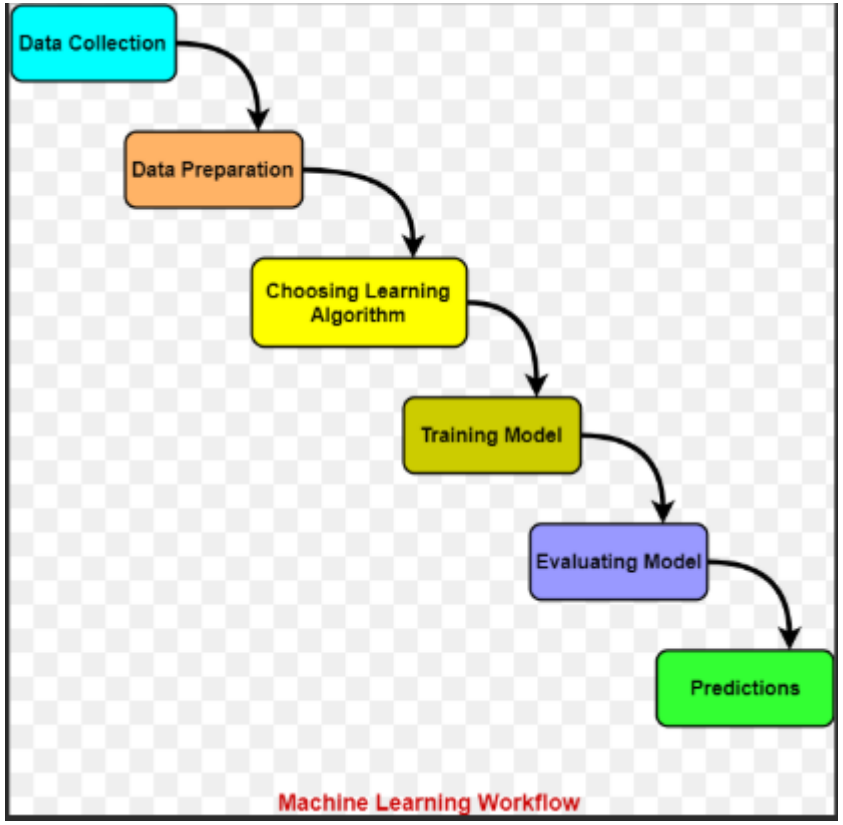
1. SGD updates model using one sample or small batch.
2. Faster than normal gradient descent.
3. Used for large datasets and deep learning.
4. Helps reduce loss step-by-step.

•**Purpose**: Optimize model parameters to **minimize a loss function**.
•**Idea**: Update parameters **step by step using small batches or single data points**, rather than the whole dataset.
•**Key Steps**:
1.Initialize parameters randomly.
2.Pick **one data point (or small batch)**.
3.Compute **gradient** of loss w.r.t parameters.
4.Update parameters: θ = θ − learning_rate × gradient
5.Repeat for many iterations/epochs.
•**Advantages**:
•Faster than full batch gradient descent
•Can escape local minima due to randomness
•**Example**: Training a neural network on large dataset using one sample at a time.



8. Explain the steps involved in Building a Machine Learning Algorithm.

- Collect data.
- Clean and preprocess data.
- Select features.
- Choose model.
- Train model.
- Validate using validation set.
- Test with unseen data.
- Deploy and monitor.



9. What are Deep Feedforward Networks?

- **Deep Forward Networks:**

1. Also called Feedforward Neural Networks.
2. Data moves from input → hidden → output layer.
3. Used for complex pattern learning.

- **Learning XOR:**

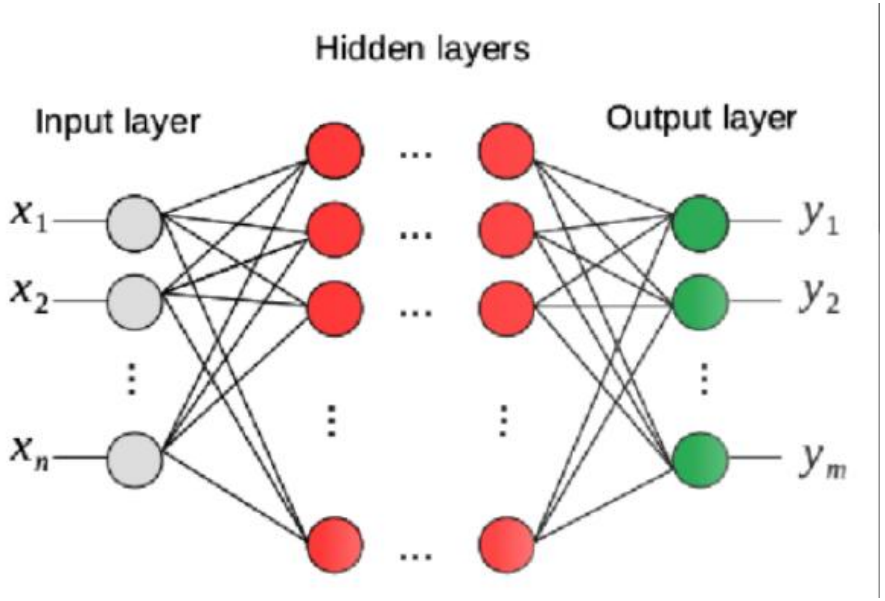
1. XOR cannot be solved by a single-layer network.
2. Requires at least one hidden layer.
3. Proves importance of deep learning.

- **Hidden Units:**

1. Neurons inside hidden layers.
2. They extract features (edges, shapes, patterns).
3. More units → more learning ability.

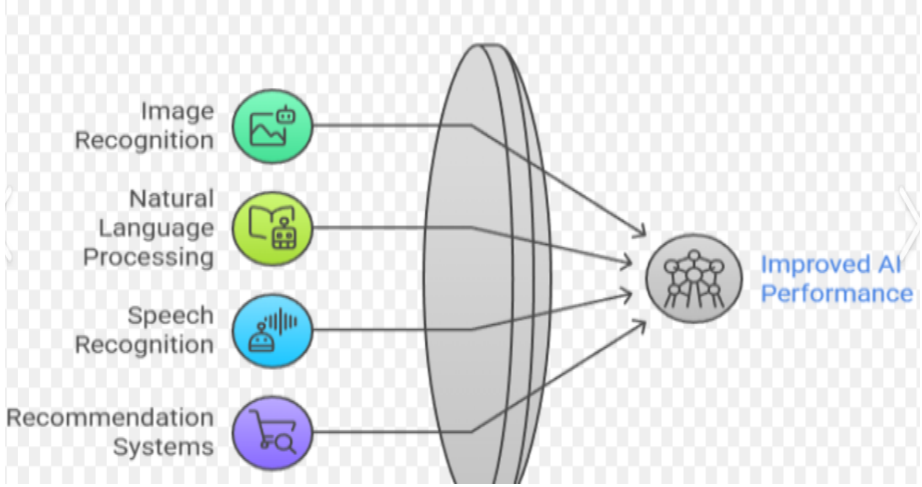
- **Architecture Design:**

1. Choosing number of layers, neurons, activation functions.
2. Good design avoids overfitting and increases accuracy.
3. Essential for deep learning models.



10. What is Gradient-Based Learning?

- Uses gradient (slope) to reduce error.
- Loss function gives direction to update weights.
- Works with gradient descent and its variants.
- Core method for training neural networks.



- **Purpose**: Optimize model parameters to **minimize a loss function**.
- **Idea**: Use the **gradient (slope) of the loss function** to guide updates to parameters.
- **Key Concept**: Move parameters in the **opposite direction of the gradient** to reduce error.
- **Steps**:
  - Initialize parameters randomly.
  - Compute gradient of loss w.r.t parameters.
  - Update parameters: $\theta = \theta - \text{learning_rate} \times \text{gradient}$
  - Repeat until loss is minimized.
- **Example**: Training neural networks using backpropagation and gradient descent.
- **Advantage**: Efficient for large and complex models like deep learning.

11. What is Backpropagation? Explain with a neat diagram.

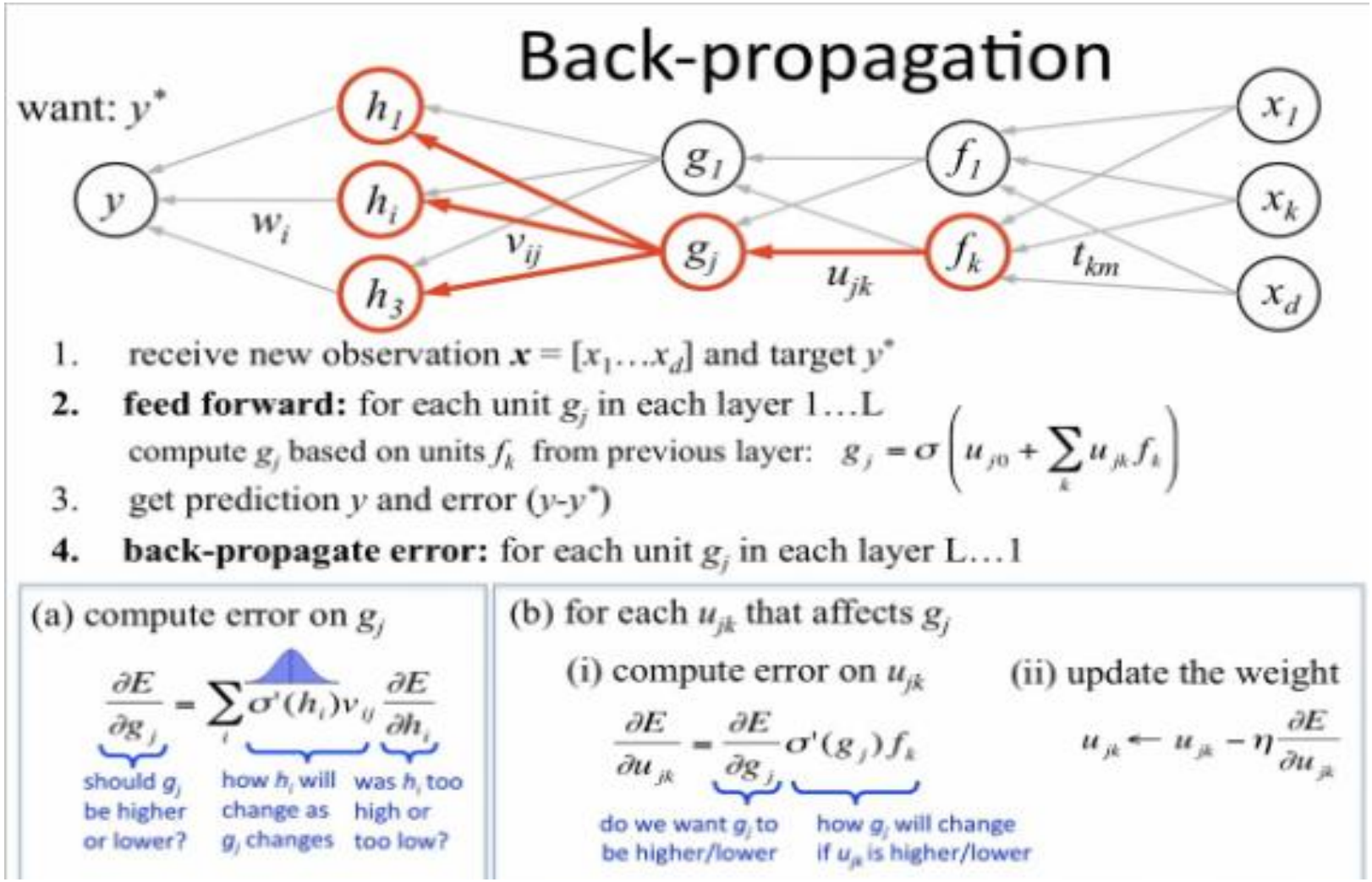
- **Back-Propagation:**

1. Calculates error at output.
2. Sends error backward through layers.
3. Adjusts weights to reduce error.
4. Works with gradient descent.

- **Other Algorithms:**

1. Momentum – speeds up learning.
2. RMSProp – stabilizes updates.
3. Adam – fast, efficient, widely used in deep learning.

Back Propagation



1.Calculates error at output.
2.Sends error backward through layers.
3.Adjusts weights to reduce error.
4.Works with gradient descent

UNIT–2: Regularization of Deep Learning
Important Questions (Solved)

Q1. Explain Parameter Norm Penalties

Answer:

1. They add an extra penalty to large weight values.
2. They control the complexity of the model.
3. They help prevent overfitting.
4. L1 penalty makes the weights sparse.
5. L2 penalty keeps weights small.
6. They help smooth the model.
7. They improve generalization performance.
8. They work during the training process.
9. They are easy to implement.
10. They are commonly used in deep learning.

Q2. Explain Norm Penalties as Constrained Optimization

Answer:

1. The size of the weights is kept within a limit.
2. Regularization is treated as a constraint in optimization.
3. The model must satisfy weight boundaries.
4. It helps reduce overfitting.
5. It keeps the model stable.
6. It improves training behavior.
7. It is similar to L1 and L2 constraints.
8. It controls the freedom of parameters.
9. It makes the model more robust.
10. It is used in many regularization techniques.

Q3. Explain Regularization and Under-Constrained Problems

Answer:

1. Under-constrained problems have too few data or too many parameters.
2. The model may memorize training data.
3. Regularization adds control to the model.
4. It reduces the freedom of the model.
5. It prevents overfitting.
6. It makes training stable.
7. It adds useful constraints.
8. It improves generalization.
9. It helps when datasets are small.
10. It is essential for deep models.

Q4. Explain Dataset Augmentation

Answer:

1. It creates more training data artificially.
2. It applies transformations such as rotation, flipping, and cropping.
3. It makes the model see data variations.
4. It reduces overfitting.
5. It improves accuracy.
6. It is cheap and effective.
7. It works for images, audio, and text.
8. It increases dataset size.
9. It makes the model more robust.
10. It is common in deep learning.

Q5. Explain Noise Robustness

Answer:

1. The model should work correctly even with noisy data.
2. Noise can be added to inputs.
3. Noise can be added to weights or activations.
4. It helps the model learn stable features.
5. It reduces overfitting.
6. It improves generalization.
7. It is useful for real-world data.
8. It prevents high sensitivity to inputs.
9. It makes predictions smoother.
10. It is used in noisy environments.

Q6. Explain Semi-Supervised Learning

Answer:

1. It uses both labeled and unlabeled data.
2. It is helpful when labeling is costly.
3. It learns patterns from large unlabeled datasets.
4. It improves accuracy.
5. It reduces manual labeling effort.
6. It is useful in NLP and vision tasks.
7. It combines supervised and unsupervised learning.
8. It improves generalization.
9. It works well with limited labeled data.
10. It is common in deep learning.

Q7. Explain Multi-Task Learning

Answer:

1. A single model is trained for multiple tasks.
2. Tasks share information with each other.
3. It prevents overfitting.
4. It improves generalization.
5. It learns common useful features.
6. It reduces overall training time.
7. It uses shared layers.
8. It helps related tasks perform better.
9. It reduces the need for large datasets per task.
10. It is useful in NLP and computer vision.

Q8. Explain Early Stopping

Answer:

1. Training is stopped before overfitting begins.
2. Validation loss is monitored.
3. Training stops when validation loss increases.
4. It prevents memorization of training data.
5. It is simple and effective.
6. It saves training time.
7. It improves generalization.
8. It reduces the need for strong regularization.
9. It works with any optimizer.
10. It is widely used.

Q9. Explain Parameter Tying and Parameter Sharing

Answer:

1. The same weights are used in multiple places.
2. It reduces the number of parameters.
3. It avoids overfitting.
4. It is used in CNNs with shared filters.
5. It is used in RNNs with shared time-step weights.
6. It makes the model compact.
7. It improves computational efficiency.
8. It helps learn consistent patterns.
9. It reduces memory usage.
10. It improves generalization.

Q10. Explain Sparse Representation

Answer:

1. Only a few neurons are active at a time.
2. It forces the model to learn key features.
3. It reduces noise.
4. It improves generalization.
5. It makes the model more interpretable.
6. It is achieved using L1 regularization.
7. It is useful in autoencoders.
8. It reduces overfitting.
9. It makes learned features meaningful.
10. It helps data compression.

Q11. Explain Bagging and Other Ensemble Methods

Answer:

1. Multiple models are trained.
2. Their predictions are combined.
3. It reduces overfitting.
4. It increases accuracy.
5. Each model sees different data subsets.
6. It makes predictions stable.
7. It works well with deep models.
8. Boosting improves weak models.
9. Stacking combines different models.
10. It is widely used.

Q12. Explain Dropout

Answer:

1. Neurons are randomly dropped during training.
2. It prevents co-dependence of neurons.
3. It reduces overfitting.
4. It makes the model robust.
5. It is a simple regularization technique.
6. It works well in deep networks.
7. It is applied to hidden layers.
8. A different network is trained each step.
9. It improves generalization.
10. It is turned off during testing.

Q13. Explain Adversarial Training

Answer:

1. Slightly modified adversarial inputs are added.
2. The model becomes robust to attacks.
3. It improves stability.
4. It improves generalization.
5. It prevents the model from being fooled.
6. It increases training difficulty.
7. It is useful in security-critical applications.
8. It works with images and text.
9. It improves model confidence.
10. It is part of robust deep learning.

Q14. Explain Tangent Distance

Answer:

1. It measures similarity between images.
2. It considers small transformations.
3. It includes rotations and shifts.
4. It helps invariant recognition.
5. It is useful in digit recognition.
6. It is more flexible than Euclidean distance.
7. It works with local changes.
8. It reduces sensitivity to variations.
9. It improves generalization.
10. It is used in classical machine learning.

Q15. Explain Tangent Prop

Answer:

1. It is a regularization technique.
2. It adds constraints using known transformations.
3. It makes the model invariant to those transformations.
4. It improves generalization.
5. It helps handle distortions.
6. It works well for image data.
7. It adds an extra loss term.
8. It guides stable feature learning.
9. It reduces overfitting.
10. It is based on tangent vectors.

Q16. Explain Manifold Tangent Classifier

Answer:

1. Data lies on a low-dimensional manifold.
2. The classifier learns tangent directions.
3. It becomes invariant to small distortions.
4. It improves accuracy.
5. It uses autoencoders to learn manifolds.
6. It helps classification tasks.
7. It reduces overfitting.
8. It learns smooth decision boundaries.
9. It uses tangent-based regularization.
10. It is suitable for image classification.

Q17. Explain Optimization for Training Deep Models

Answer:

1. Deep models require strong optimization techniques.
2. Proper initialization is needed.
3. Advanced optimizers like Adam and RMSProp are used.
4. Batch normalization is applied.
5. Learning rate schedules are used.
6. Dropout and regularizers are applied.
7. Large batch training is used.
8. Vanishing gradients are avoided.
9. Momentum is used.
10. It helps achieve best accuracy.

Q18. Explain Learning vs Pure Optimization

Answer:

1. Optimization minimizes loss functions.
2. Learning extracts patterns from data.
3. Good optimization does not guarantee good learning.
4. Models should generalize, not memorize.
5. Regularization supports learning.
6. Over-optimization causes overfitting.
7. Learning focuses on meaningful patterns.
8. Optimization focuses on mathematical minimization.
9. Balance is required in deep learning.
10. Both work together.

Q19. Explain Challenges in Neural Network Optimization

Answer:

1. Vanishing gradient problem.
2. Exploding gradient problem.
3. Saddle points.
4. Local minima.
5. Flat regions in loss surface.
6. High computational cost.
7. Large number of parameters.
8. Difficulty in choosing hyperparameters.
9. Overfitting issues.
10. Long training time.

Q20. Explain Basic Algorithms

Answer:

1. Gradient Descent.
2. Stochastic Gradient Descent.
3. Mini-batch Gradient Descent.
4. Momentum method.
5. RMSProp.
6. Adam optimizer.
7. AdaGrad.
8. Learning rate scheduling.
9. Backpropagation algorithm.
10. Weight updates using gradients.

Q21. Explain Parameter Initialization Strategies

Answer:

1. Proper initialization is crucial.
2. It avoids vanishing and exploding gradients.
3. Xavier initialization is used.
4. He initialization is used.
5. Random small values are assigned.
6. Zero initialization is avoided for weights.
7. Biases are set to small values.
8. It enables faster training.
9. It improves convergence.
10. It is essential for deep networks.

Q22. Explain Algorithms with Adaptive Learning Rates

Answer:

1. Learning rates change automatically.
2. They provide faster convergence.
3. AdaGrad adapts learning rate per parameter.
4. RMSProp fixes AdaGrad limitations.
5. Adam combines RMSProp and momentum.
6. They reduce manual tuning effort.
7. They ensure stable training.
8. They work well for deep networks.
9. They handle sparse gradients effectively.
10. They are the most popular optimizers.

Unit 3
Convolutional Networks

- **Important Questions:**

1. Explain **The Convolution** with **Operations and Motivation**.
2. What is **Pooling**?
3. Explain **Convolution and Pooling as an Infinitely Strong Prior**.
4. Explain the **Variants of Basic Convolution Function**.
5. What is **Structured Output** in Convolutional Networks?
6. Explain the importance of **Data Type** in CNNs.
7. Explain **Efficient Convolution Algorithm**.
8. What are **Random or Unsupervised Features** in CNNs?

1.Explain The Convolution with Operations and Motivation.

•**Definition:** Convolution is a specialized linear operation where a small matrix (called a *kernel* or *filter*) is applied to input data to produce a transformed output.

•**Process:**

1. Place the kernel on a section of the input.
2. Multiply corresponding values element-wise.
3. Sum the results to produce a single output value.
4. Slide the kernel across the input and repeat.

- **Result:** A new matrix (called a *feature map*) that highlights specific patterns in the input.

Operations in Convolution

- **Sparse Connectivity:** Each output depends only on a small region of the input, unlike fully connected layers.
- **Parameter Sharing:** The same kernel is used across the entire input, reducing the number of parameters.
- **Equivariance to Translation:** If the input shifts, the output shifts in the same way, making convolution effective for detecting patterns regardless of position.
- **Pooling (often paired with convolution):** Reduces dimensionality by summarizing regions (e.g., max pooling takes the maximum value in a region).

Variants:

1D convolution (for time-series data)

2D convolution (for images)

3D convolution (for videos or volumetric data)

Motivation Behind Convolution:

Efficiency: Convolution drastically reduces the number of parameters compared to fully connected layers, making models faster and less prone to overfitting.

Feature Extraction: Kernels can learn to detect edges, corners, textures, and complex shapes in images.

Biological Inspiration: CNNs are loosely inspired by the human visual cortex, where neurons respond to local receptive fields.

Scalability: Convolution allows deep networks to process large inputs (like high-resolution images) without exploding computational costs.

Generalization: By focusing on local patterns and sharing parameters, convolution helps models generalize better across unseen data.

2. What is Pooling?

- Pooling is a step in **Convolutional Neural Networks (CNNs)** that reduces the size of feature maps while keeping the most important information.
- **Types:**
  - **Max Pooling:** keeps the largest value in each region (most common).
  - **Average Pooling:** takes the average of values in the region.
  - **Global Pooling:** summarizes the entire feature map into one value.
- **Why it's useful:** It makes models faster, prevents overfitting, and helps recognize patterns regardless of their exact position.
- **Pooling shrinks feature maps but keeps the strongest features, making CNNs efficient and robust.**

3. Explain Convolution and Pooling as an Infinitely Strong Prior.

What Does “Infinitely Strong Prior” Mean?

In machine learning, a **prior** is an assumption about the structure of data before learning begins.

An **infinitely strong prior** means the model is *forced* to follow that assumption — it cannot learn otherwise.

So, convolution and pooling aren’t just “choices”; they **hard-code assumptions** about how data should be processed.

- **Convolution as an Infinitely Strong Prior:**

Locality Assumption: Convolution assumes that meaningful patterns are local (edges, textures). The kernel only looks at small neighborhoods.

Translation Equivariance: If an object shifts in the input, convolution ensures the feature map shifts accordingly.

Parameter Sharing: The same kernel is applied everywhere, enforcing the belief that the same features matter across the whole input.

Infinitely Strong Prior: The network *must* learn features in this way — it cannot decide to connect every pixel to every other pixel like a fully connected layer.

Pooling as an Infinitely Strong Prior:

- **Spatial Invariance Assumption:** Pooling assumes that the exact position of a feature doesn't matter, only its presence.
- **Dimensionality Reduction:** By summarizing regions (max or average), pooling enforces the idea that fine-grained location details are less important.
- **Infinitely Strong Prior:** The network is *forced* to ignore small shifts or distortions, because pooling discards precise positional information.

Why This Matters:

- These priors make CNNs **efficient and generalizable** for vision tasks.
- They reduce parameters, prevent overfitting, and align with how human vision works (local receptive fields, invariance to small shifts).
- But they also **limit flexibility**: if the task requires global relationships or exact positions, convolution + pooling may struggle.

4. Explain the Variants of Basic Convolution Function.

- **Multichannel Convolution:**
- Works with multi-inputs (e.g., RGB).
- Kernel has weights per channel.
- Outputs combined feature map.

- 🔹 **Strided Convolution:**
- Kernel moves in larger steps.
- Shrinks feature map size.
- Faster, less computation.

- 🔹 **Locally Connected Layers:**
- No weight sharing.
- Each region has unique filters.
- More flexible, but heavy.

- **Tiled Convolution**
- Partial weight sharing.
- Filters repeat in tiles.
- Balance of efficiency + variation.

- **Dilated Convolution**
- Kernels with gaps.
- Enlarges receptive field.
- Captures wider context.

- **Transposed Convolution**
- Reverse of convolution.
- Expands feature maps.
- Used for upsampling/reconstruction.

- **Multichannel = color, Stride = speed, Local = unique, Tiled = partial, Dilated = wide, Transposed = upsample.**

5. What is Structured Output in Convolutional Networks?

- 🔹 **What is Structured Output:**
- Output is not a single label, but a **map/sequence/set**.
- Predictions are **interdependent** (pixels, boxes, words).
- Used in tasks like segmentation, detection, captioning.

- 🔹 **Why It's Needed:**
- Many problems need **multiple labels** per input.
- Captures **relationships** among outputs (spatial/temporal).
- Improves **accuracy + context awareness**.

- 🔹 **How CNNs Handle It:**
- **Fully Convolutional Networks (FCNs):** output pixel-wise maps.
- **CNN + RNN/CRF:** model sequences or enforce consistency.
- **Detection heads:** predict bounding boxes + classes together.

- Structured output lets CNNs produce rich, context-aware predictions (maps, sequences, sets) instead of just one label.

6. Explain the Data Type in CNNs.

- **1D Data (Sequences)**
- **Examples:** Text, audio signals, time-series data.
- **Operation:** 1D convolution slides kernels along one dimension.
- **Goal:** Detect local patterns like word n-grams in text or phonemes in audio.

- 🔹 **2D Data (Images)**
- **Examples:** Grayscale or RGB images.
- **Operation:** 2D convolution applies kernels over height × width.
- **Goal:** Capture spatial features like edges, textures, and shapes.

- 🔹 **3D Data (Videos / Volumetric Data**)
- **Examples:** Video frames, medical scans (MRI, CT).
- **Operation:** 3D convolution spans height × width × time/depth.
- **Goal:** Learn spatiotemporal features (motion in video, 3D structures in scans).

- 🔹 **Multi-Channel Data**
- **Examples:** RGB channels in images, multi-sensor inputs.
- **Operation:** Kernels process each channel separately, then combine.
- **Goal:** Integrate diverse information (color, depth, modalities).

1D CNNs → sequences (text/audio).

2D CNNs → images.

3D CNNs → videos/volumes.

Multi-channel → combine multiple inputs.

7. Explain Efficient Convolution Algorithm

- 1. **Why Needed**
- Convolution is computationally heavy (millions of multiplications).
- Large CNNs require optimization for speed and memory.
- Efficiency enables real-time tasks and mobile deployment.
- 2. **Main Techniques**
- **FFT Convolution:** Converts convolution into multiplication in frequency domain → faster for large kernels.
- **Winograd Convolution:** Reduces number of multiplications for small kernels (e.g., 3×3).
- **Separable Convolution:** Breaks convolution into depthwise + pointwise → fewer parameters, lightweight models.
- **Group Convolution:** Splits channels into groups → parallel, less computation.
- **Dilated Convolution:** Adds gaps in kernels → larger receptive field without extra cost.
- **Transposed Convolution:** Used for upsampling → efficient reconstruction.
- 3. **Importance**
- Speeds up training and inference.
- Reduces energy and memory usage.
- Makes CNNs practical for mobile, IoT, and real-time applications.

8. What are Random or Unsupervised Features in CNNs?

- **Random Features**
- Filters initialized randomly, not trained.
- Still capture basic edges/textures due to locality.
- Used as baseline or lightweight models.

- 🔹 **Unsupervised Features**
- Learned without labeled data.
- Methods: clustering, autoencoders, contrastive learning.
- Useful when labeled data is scarce.

- 🔹 **Importance**
- Saves labeling effort.
- Improves generalization across tasks.
- Supports pretraining before supervised fine-tuning.

- **Random = untrained filters, Unsupervised = learned without labels, Importance = efficiency + generalization.**

Unit 4 – Important Questions
(Solved & Explained Based on Marks)

Q1. Explain Unfolding Computational Graphs

Answer:

1. RNNs work step-by-step on sequence data.
2. Unfolding means expanding the RNN across time steps.
3. Each time step becomes a separate layer in the graph.
4. It shows how information flows over time.
5. It converts a loop into a long chain of operations.
6. It makes backpropagation through time possible.
7. It helps visualize hidden state updates.
8. It shows how outputs depend on previous states.
9. It helps detect long-term dependency issues.
10. It is useful for training and debugging RNNs.

Q2. Explain Recurrent Neural Networks

Answer:

1. RNNs are used for sequence data.
2. They have feedback connections.
3. They use previous state and current input.
4. They can remember past information.
5. They are useful for text, speech, and time series data.
6. The same weights are used at all time steps.
7. They can handle variable-length input.
8. They are hard to train for long dependencies.
9. They face the vanishing gradient problem.
10. They are improved by LSTM and GRU.

Q3. Explain Bidirectional RNNs

Answer:

1. They process sequence input in the forward direction.
2. They also process the sequence in the backward direction.
3. Outputs from both directions are combined.
4. They capture both past and future context.
5. They are useful for NLP tasks.
6. They give better accuracy than normal RNNs.
7. They need the full sequence beforehand.
8. They are not suitable for real-time prediction.
9. They require more computation.
10. They are used in speech recognition and text tasks.

Q4. Explain Encoder–Decoder Sequence-to-Sequence Architecture

Answer:

1. It is used for mapping one sequence to another.
2. The encoder reads the input sequence.
3. It converts the input into a fixed-size vector.
4. The decoder uses this vector to generate output.
5. It works with RNN, LSTM, or GRU models.
6. It handles variable input and output lengths.
7. It generates output one step at a time.
8. It is useful for machine translation.
9. Attention mechanisms improve output quality.
10. It is used in chatbots and text summarization.

Q5. Explain Deep Recurrent Networks

Answer:

1. They consist of multiple stacked RNN layers.
2. Each layer learns different features.
3. They capture complex sequence patterns.
4. They provide higher accuracy.
5. They require large datasets for training.
6. They are harder to train.
7. They are good for speech and language tasks.
8. They involve more parameters.
9. They need regularization techniques.
10. They are often combined with LSTM or GRU.

Q6. Explain the Challenge of Long-Term Dependencies

Answer:

1. Some tasks require remembering information from far in the past.
2. Basic RNNs forget long-past inputs.
3. Gradients become very small during training.
4. This makes learning long-distance patterns difficult.
5. Training becomes unstable.
6. It causes poor model accuracy.
7. It affects tasks like translation and speech recognition.
8. LSTM solves this problem using gates.
9. GRU also helps overcome this issue.
10. Attention mechanisms help capture long-term relationships.

Q7. Explain Recursive Neural Networks

Answer:

1. They work on tree-structured data.
2. They combine inputs in a hierarchical manner.
3. They are useful for parsing sentences.
4. They capture part–whole relationships.
5. They build representations bottom-up.
6. They share weights at every merge step.
7. They are good for syntactic NLP tasks.
8. They are harder to design.
9. They are slow for very deep trees.
10. They are used in sentiment analysis.

Q8. Explain Echo State Networks

Answer:

1. They are a type of RNN with fixed random weights.
2. Only the output weights are trained.
3. This makes training very fast.
4. They contain a large reservoir of dynamic states.
5. They are good for time-series prediction.
6. They avoid the vanishing gradient problem.
7. They work well with small datasets.
8. They are simple to implement.
9. Performance depends on reservoir quality.
10. They are used in control and signal processing tasks.

Q9. Explain Leaky Units and Other Strategies for Multiple Time Scales

Answer:

1. Leaky units update their states slowly.
2. They retain old information for longer durations.
3. They capture both long-term and short-term patterns.
4. They improve the stability of RNNs.
5. They reduce the vanishing gradient problem.
6. They are useful for slow-changing signals.
7. They support learning at multiple time scales.
8. They improve overall model performance.
9. They are often combined with gated units.
10. They are used in advanced recurrent models.

Q10. Explain LSTM and Other Gated RNNs

Answer:

1. LSTM solves the vanishing gradient problem.
2. It uses gates to control information flow.
3. The input gate adds new information.
4. The forget gate removes unnecessary information.
5. The output gate controls the final output.
6. GRU is a simpler gated RNN.
7. Both handle long sequences effectively.
8. They are widely used in NLP and speech tasks.
9. They improve accuracy on long data sequences.
10. They are the most popular RNN architectures.

Q11. Explain Optimization for Long-Term Dependencies

Answer:

1. Optimization is needed for training long sequences.
2. LSTM and GRU help avoid gradient problems.
3. Gradient clipping prevents exploding gradients.
4. Proper weight initialization is important.
5. Adam and RMSProp optimizers are commonly used.
6. Dropout is applied for regularization.
7. Attention helps focus on important inputs.
8. Learning rate tuning improves convergence.
9. Batching and truncation methods are used.
10. These techniques help learn long-term patterns effectively.

Q12. Explain Explicit Memory Networks

Answer:

1. They add external memory to neural models.
2. The model can read information from memory.
3. The model can write information into memory.
4. They store information for long durations.
5. They are useful for reasoning tasks.
6. They reduce forgetting of important information.
7. They are used in memory network architectures.
8. They help in question answering systems.
9. They support complex sequence learning tasks.
10. They improve long-term information storage.

Unit 5

1. Explain Performance Metrics in Deep Learning.
2. What are Default Baseline Models? Explain with examples.
3. How do we determine whether to gather more data in a deep learning problem?
4. Explain the process of selecting hyperparameters in deep learning.
5. Describe the debugging strategies used in deep learning model training.
6. Explain the example of Multi-Digit Number Recognition.
7. Write the applications of Deep Learning.

1. Explain Performance Metrics in Deep Learning.

- Performance metrics tell how good a model is.
- Accuracy shows how many predictions are correct.
- Loss shows how wrong the model is.
- Precision tells correct positive predictions.
- Recall tells how many real positives are found.
- F1-Score is the balance of precision and recall.
- MSE/MAE measure errors in regression.
- Confusion Matrix shows correct and wrong predictions.
- These metrics help compare models.
- They guide us to improve the model’s performance.

2. What are Default Baseline Models? Explain with examples.

•DBM means Default Baseline Models.
•They are very simple models used as the first reference.
•They show the minimum performance we should achieve.
•If our deep learning model cannot beat DBM, the model is useless.
•They help us compare, check errors, and improve.
•Example (Classification): Predict the most common class always.
•Example (Regression): Predict the average value for all inputs.
•DBM is used to check whether building a complex model is worth it.

•**Classification Example**
If 80% of students passed and 20% failed,
DBM will always predict “Pass” for everyone.
This gives 80% accuracy as a baseline.

•**Regression Example**
If the average house price in the dataset is 40 lakhs,
DBM will always predict “40 lakhs” for every house.

3. How do we determine whether to gather more data in a deep

1. If model is underfitting.
2. If test accuracy is low.
3. If model stops improving.
4. If loss is not decreasing well.
5. If predictions are unstable.
6. If problem is complex (images, text, speech).
7. If small data changes give big output changes.
8. If deep model is not beating baseline.
9. If dataset is noisy.
10. If validation accuracy increases with more data.

4. Explain the process of selecting hyperparameters in deep learning.

1. Hyperparameters control how a model learns.
2. Examples: learning rate, batch size, epochs.
3. They are set before training, not learned.
4. Start with common default values.
5. Choose a range of values to test.
6. Use a validation set to check performance.
7. Try grid search or random search.
8. Use Bayesian optimization for smart selection.
9. Compare results using accuracy or loss.
10. Pick the combination with the best results.

5. Describe the debugging strategies used in deep learning model training.

1. **Check data quality** – ensure no missing or wrong labels.
2. **Visualize data** – understand patterns and detect anomalies.
3. **Simplify the model** – start small to test basic learning.
4. **Use small batches** – helps isolate problems quickly.
5. **Monitor loss and metrics** – track training and validation trends.
6. **Check for overfitting** – compare training vs validation performance.
7. **Test with known outputs** – see if the model can predict simple cases.
8. **Gradient checking** – ensure backpropagation is working correctly.
9. **Use logging and checkpoints** – track changes and restore models.
10. **Experiment systematically** – change one factor at a time and observe effects.

6. Explain the example of Multi-Digit Number Recognition.

1. Multi-digit number recognition identifies numbers with more than one digit from images.
2. Example: reading house numbers from street images.
3. Input is an image containing digits (like “123”).
4. Preprocess the image – resize, normalize, and convert to grayscale.
5. Use a Convolutional Neural Network (CNN) to extract features from digits.
6. CNN outputs feature maps representing digit patterns.
7. Flatten features and use a sequence model (like RNN or CTC) to read multiple digits.
8. The model predicts each digit in order.
9. Compare predictions with true labels to calculate accuracy.
10. Applications include postal code reading, license plate recognition, and bank cheque processing.

7. Write the applications of Deep Learning.

1. **Image Recognition** – detecting objects, faces, and handwriting.
2. **Speech Recognition** – converting speech to text.
3. **Natural Language Processing (NLP)** – translation, chatbots, sentiment analysis.
4. **Autonomous Vehicles** – self-driving car navigation and obstacle detection.
5. **Medical Diagnosis** – detecting diseases from X-rays, MRIs, or scans.
6. **Recommender Systems** – suggesting products, movies, or music.
7. **Finance** – fraud detection and stock market prediction.
8. **Robotics** – controlling robots and performing complex tasks.
9. **Gaming and AI** – smart game-playing agents.
10. **Virtual Assistants** – Siri, Alexa, and Google Assistant tasks.

- **Large-scale Deep Learning** – training big models for tasks like language models and AI research.
- **Computer Vision** – image recognition, object detection, medical imaging, autonomous vehicles.
- **Speech Recognition** – converting speech to text, voice assistants.
- **NLP (Natural Language Processing)** – translation, chatbots, sentiment analysis.
- **Other Applications** – recommender systems, robotics, finance (fraud detection, prediction), gaming AI.